

OLC

Placement Assessment

How it was built, and how the whole thing works. A handover document for the engineer picking this up.

Repository	jupiter133/jupiter
Branch	claude/olc-placement-assessment-y4lrpw
Stack	React 18 + TypeScript + Vite. Plain CSS, no UI framework. Vitest.
Bank	741 items across 14 subjects, one JSON file
Tests	157, all passing
State	No placeholder content remains. One thing is unbuilt — see §10.

Read §10 first if you read nothing else. Nothing scores a spoken answer. Three subjects produce observations that move no placement, and two of them are load-bearing. That is the one open decision in this project.

Contents

§	Section	§	Section
1	What this is	6	The 14 subjects
2	The flow, screen by screen	7	Answer modes
3	Two tracks, one product	8	Layout rules that cost us
4	The adaptive engine	9	Testing and verification
5	The priority gate	10	What is NOT built
		11	Running it

1. What this is

A placement assessment that a child sits once, on a tablet, to find the OLC program they belong in. A grown-up sets it up, hands the device over, and gets a result screen at the end. The child never sees a score.

Three principles that shaped every decision

Principle	What it means in the code
Correctness-blindness	There is no red / error colour role in the design system at all. Nothing is ever marked wrong mid-flow, and the child is never shown a score. A chosen answer just looks chosen.
Grade places, age presents	Grade drives which tier a child starts at and what the result means. Age drives wording, art, read-aloud defaults and which track they sit. The two are never conflated.
Never guess a verdict	An answer nothing could judge is recorded as <code>scored: false</code> . It moves no streak and no tier, and the parent is told it was not measured. A fabricated number would be worse than a gap.

Module map

Path	Holds
<code>src/assessment/types.ts</code>	Every shared type, plus the predicates the screens branch on (<code>isListenQuestion</code> , <code>isOddQuestion</code> , ...). Screens ask a predicate, never a subject name.
<code>src/assessment/engine.ts</code>	Question selection, branching, stop rule. Pure functions over a session state.
<code>src/assessment/gate.ts</code>	The five ordered rules that turn subject results into a program.
<code>src/assessment/tiers.ts</code>	Tier ↔ grade mapping, gap maths, the behind-threshold.
<code>src/assessment/usePlacementFlow.ts</code>	The one hook that owns the whole flow: which screen, which sitting, what happens on skip.
<code>src/content/questionBank.json</code>	All 741 items. Subject and tier live in the data, so replacing content needs no engine change.
<code>src/components/</code>	One component per answer mode — drag, order, write, keypad, match, pair.
<code>src/screens/</code>	One per stage of the flow. <code>QuestionScreen.tsx</code> is the hub.
<code>src/components/glyphs.tsx</code>	39 illustrations, each drawn in a 100×100 box using the locked colour roles.

2. The flow, screen by screen

#	Screen	Who is holding the device
1	Start	Grown-up. Name and age.
2	Parent context	Grown-up. Grade, and a mismatch check against age.
3	Handoff	Grown-up → child. Shows the seven sections and hands over.
4	Section intro	Child. What this section is, in the track's own words.
5	Question	Child. The hub — renders every answer mode.
6	Section complete	Child. No score, just progress.
7	Kid complete	Child. Coins, then hand back.
8	Parent results	Grown-up. Program, per-subject rows, what was not measured.

Deferring a section

Any section can be skipped with “Do this one later”. This was the source of the worst bug in the project: skipping cleared the session but built no sitting for the next subject, so pressing Start rendered a question screen with no question — a blank white page, in a product for four-year-olds. `doThisLater` now builds the next sitting, `startSection` is defensive, and a sweep (`noBlank.mjs`) walks every skip combination on both tracks asserting the stage is never empty.

Read-aloud

- Defaults ON for the whole Little Readers track and for the junior band. The default cannot be silence for a child who cannot read.
- A wordless option is never narrated. It used to read “A. B. C.” over picture cards, burying the one line that said what to do. A test holds this.
- Listen items (Spelling, the heard third of Letter Sounds) get a replay counter, currently 3.

3. Two tracks, one product

A five-year-old and a ten-year-old are not doing the same thing, so they do not sit the same activities. Age chooses.

`trackFor (age, grade)` — $\text{age} \leq 6$ sits **Little Readers**, $\text{age} \geq 7$ sits **Grade Level**. Grade is the fallback only when an account never captured an age. The boundary is one constant, `LITTLE_READER_MAX_AGE`, because it is a judgement about children rather than a fact about code. **Pending teacher sign-off.**

	Little Readers	Grade Level
Age	≤ 6	≥ 7
Sections	Find the Same, Match Making, Spot the Difference, Shapes & Colors, Number Fun, Letter Sounds, Word Practice	Words Speaking, Oral Reading, Vocabulary, Reading Comprehension, Spelling, Sentence Writing, Math
Tone	A park. Sections are “rides”.	Plain language. Sections are “sections”. The tower cosplay was removed — a ten-year-old told they are ascending a tower of wisdom knows they are being managed.
Places on	Letter Sounds and Word Practice only. The other five are readiness observations.	Oral Reading and Reading Comprehension .

The rest of each track is recorded but does not decide the placement. `readingSubjectsFor (track)` is the single source of that, and a test asserts every track only rests on subjects it actually sits.

4. The adaptive engine

Tiers

Tier 0 = Kindergarten ... tier 8 = Grade 8. A child starts at the tier matching their grade. The tier a sitting *ends* on is the placement for that subject; the gap between that and their grade tier is what the gate reads.

Branching

STREAK_TO_MOVE = 2. Two correct in a row moves up a tier, two wrong moves down. Either move resets *both* streaks, so a fresh pair is needed at the new level before moving again.

Stopping

STABILITY_WINDOW = 4. When the tier has not changed across the trailing four answers, the sitting ends — usually after five or six questions. **This is the single most important fact about the content**, and it caught us out three times: a child who guesses never leaves the tier they start on, so anything you put two tiers up is content most children will never see. The window is suspended for a sitting where every answer is unscored, so a spoken section cannot end early on no evidence.

Choosing the next question

Among items equally close to the current tier, the selector prefers variety, at two levels that are not interchangeable:

Level	Why it comes first
1. Unused answer mode	A maths sitting that never serves the typed-number or ordering items lets a child work every answer backwards from the options. This is the level that protects validity.
2. Unused skill	Two items can share a mode and still be different questions — naming a shape, naming a colour and painting a shape are all tapped. Keying on mode alone served whichever came first in the bank, so a child met shapes and nothing else in the ride named for both.

Variety never costs a child a question at the wrong level: the selector only reaches past the nearest tier when the fresher item is exactly as close.

Unscored answers

`submitAnswer(state, id, { scored: false })` records the answer and does nothing else — no streak, no tier move. `SubjectResult.unscored` carries it to the results screen, where it renders as “*Recorded — for review*” rather than a level.

5. The priority gate

Five rules, evaluated in order, first match wins. **The order is the policy.** A child who cannot read is placed on reading no matter what the later subjects say, because those results measure reading as much as they measure their own subject — a maths word problem is a reading test wearing a hat.

#	Condition	Outcome
1	Reading below a Grade 3 level (absolute, not a gap)	Reading track
2	Reading more than one grade behind	Core Skills Reading
3	Reading within one grade, writing or spelling behind	Core Skills Writing
4	Literacy all within one grade, math behind	Core Skills Math
5	All four within one grade	Enriched

Rules 3 and 4 restate their predecessors' conditions so each is true on its own terms. That is what makes the hard blocks hold even if someone reorders the list later. An exhaustive sweep asserts them.

`evaluateGate` returns `null` when a rule needs evidence that has not been gathered — the flow then asks for it rather than guessing. `determinedBy` names the subjects the decision actually rested on; everything else is flagged non-determining on the results screen, so a parent is never shown a number that did not matter.

6. The 14 subjects

Every subject was built the same way: decide what it actually measures, write a difficulty ladder, generate the items with a script that asserts its own rules, then add tests that hold the content honest, then look at it in a real browser at four viewports.

Little Readers — 741 items total, 399 here

Section	n	What the child does	The ladder
Find the Same	54	Tap the two pictures that match. Keyed by the <i>pair</i> of ids.	More cards (3→6), then closer pairs — at the top, two counts of the same object differing by one.
Match Making	54	Move each picture onto its twin. Drag or tap; tap is always an equal path.	More pairs, then near-identical pieces.
Spot the Difference	54	Tap the card that does not belong. Shares a component with Find the Same via a pick prop — same act, different count.	Different object → different count → count off by one.
Shapes & Colors	75	Name a shape, name a colour, or paint an outlined shape from a palette. No labels on the cards — a card reading “Diamond” makes it a reading test.	Both shapes and colours at every tier; only the top rung asks for both at once.
Number Fun	54	Find a written number, find a group to count, or count a group and tap the number. The third is the bridge and the one that matters.	The size of the numbers — to 3, to 5, to 10 — then reasoning at 6–8.
Letter Sounds	54	Hear a sound and find the picture, see a letter and find the picture, see a picture and find the letter. The sound is played, never printed.	Which sounds: m s b t f p, then the rest, vowels last.
Word Practice	54	Read a word and find the picture, see a picture and find the word, read a word aloud.	Word length. Three letters, then four, then <i>apple</i> , then <i>snowflake</i> .

Grade Level — 342 items

Section	n	What the child does	Notes
Words Speaking	54	Says a word into the microphone.	Unscored. See §10.
Oral Reading	27	Reads a short passage aloud, with read-along bolding.	Unscored , and it is half the reading gate.
Vocabulary	45	Studies a word, then recalls its meaning without it on screen.	Study → confirm → recall.
Reading Comprehension	54	Studies a passage, then answers with the passage gone.	11 question types. No passage is shared with Oral Reading — tests enforce it, because reading the same text twice makes one a rehearsal and the other a memory check.
Spelling	63	Hears a word, never sees it, picks the spelling. One item is a whole spoken sentence with a drag-and-drop gap.	A voiceless device makes these unscored rather than guessed.
Sentence Writing	45	Orders words into a sentence, builds one from a heard sentence, or types one.	markSentence checks four objective things: required words present, opening capital, terminal punctuation, longer than the words handed.
Math	54	Five shapes: tap, typed keypad, ordering, picture maths, missing number.	Four tappable options can be worked backwards, so the typed and ordered shapes exist to stop that.

Two content rules worth keeping

Topic is not difficulty

Shapes & Colors first put colours two tiers above shapes. Because a sitting stops after five or six questions and a guessing child never climbs, the ride named for both showed nothing but shapes. Measured, three ways of playing:

before after
 all right shape 2 colour 3 shape 3 colour 2
 all wrong shape 5 colour 0 shape 2 colour 2
 random shape 5 colour 0 shape 3 colour 2

A three-year-old knows “red” long before “rectangle”. Difficulty now grows *inside* each topic — more cards, harder shapes, bigger numbers — never by swapping the topic out. Adding within ten at the top of Number Fun is fine, because that genuinely is harder.

A picture is only usable if a child names it one way

The sharpest risk in Letter Sounds, and no test of the mechanics would ever catch it. A picture is unusable when the *other* name a child would reach for starts with a different sound that is also in the answer set — the item then marks a child wrong for naming the picture sensibly. Twelve words were cut:

Picture	Also called	Which is	Picture	Also called	Which is
moose	“deer”	/d/	bottle	“tube”	/t/
mitten	“glove”	/g/	map	“paper”	/p/
river	“water”	/w/	rope	“knot”	/n/
gift	“present”	/p/	van	“truck”	/t/
bowl	“soup”	/s/	compass	“circle”	/s/
pinecone	“acorn”	/a/	berry	“grapes”	/g/

Cutting them cost /v/ its only word, so v is out of the sound set. **A smaller alphabet that measures something beats a full one that measures whether a child shares the author’s vocabulary.** A test bans all twelve, from the answer cards and from the panel.

7. Answer modes

`Question.answerMode` decides which component renders. Screens branch on predicates in `types.ts`, never on a subject name, which is why a new subject needs no screen change.

Mode	Component	Notes
tap	inline in QuestionScreen	The default. Picture-only options get their own wordless layout.
drag	DragAnswer	One tile into one gap.
order / order-drag	OrderAnswer	Every tile into a sequence. Tap always works as an equal path.
write	WriteAnswer	Free text, marked by rubric.
number	NumberAnswer	On-screen keypad.
match / odd	MatchAnswer	One component, a pick prop of 2 or 1.
pair	PairAnswer	Each picture onto its twin.

Dragging is pointer-events, not HTML5 drag-and-drop, which does not fire on touch. Exactly one dragged item is served per sitting (`DRAW_QUESTION_POSITION`), and every draggable answer also has a tap path — a child who cannot aim must never be blocked.

8. Layout rules that cost us

These are written down because each one shipped wrong at least once, and one of them three times.

Check what the child can see, not what the DOM says

Three separate bugs hid behind a page that did not scroll: paint chips, counting cards and text answers each ran past the bottom of a short screen. The stage does not scroll, so the last answer was not “below the fold”, it was **unreachable** — and the viewport audit reported `overflow: 0px` every time. The browser scripts now measure the last answer’s bottom edge against the viewport, and for counting cards compare the number a card claims against the glyphs whose box actually sits inside it.

Size off the constrained dimension

- A percentage width against an auto-width wrapper silently collapses to nothing.
- `margin-inline: auto` **stops a flex item stretching**. A grid shrink-wrapped to 163px inside a 763px parent and the cards came out 76px square. Use `width: 100%` plus `align-self: center`.
- `grid-auto-rows: minmax(min-content, 1fr)` and a square card are circular — the square’s min-content floor is its own width, so the grid grows past its box. Use `1fr` with `min-height: 0`.
- An `<svg>` with inline width/height attributes ignores `aspect-ratio`. Picture cards use a container query: `width: min(76cqw, 76cqh)`.
- The card fills its cell; the **picture** is the square. Chasing a square card means fighting the grid, and one of the two always wins by clipping.

Then measure it in a browser, at every viewport, rather than trusting it.

9. Testing and verification

157 unit tests, and a set of Playwright scripts that drive the real build. The unit tests mostly guard *content*, not code — a bank is where an assessment goes wrong quietly.

What the content tests assert

- Exactly one option can be right. Two pictures starting with the same sound, or two identical cards in an odd-one-out, make an item with two answers.
- The question never prints its own answer. The stub Number Fun bank read “How many stars are there? (1)” with “Count them: 1.” underneath.
- The answer is never parked in one position within a tier and a move. A child who notices the answer is always last stops reading the question.
- Distractors are plausible: counts within four of the answer, misreadings within two letters. “cat” against “elephant” is a length question.
- Red and green never decide an item — specifically, the *answer* is never opposite the other one. A colour-blind child is assessed on what they know.
- Every subject uses one format throughout. Old items survived a rewrite twice; this catches it.
- No passage appears in two subjects.
- The bank’s own note names exactly the subjects still carrying placeholder items. It went stale the moment a stub was replaced, which is the kind of claim nobody rechecks.

Browser scripts

Script	What it proves
noblank.mjs	Walks every skip combination on both tracks and asserts the stage is never empty. This is the one that would have caught the white screen.
audit5.mjs	Six profiles × every screen: horizontal overflow, gutters, unexpected scrolling.
nfcount.mjs	Every counting card shows its whole count — measured, not counted in the DOM.
sc-grid.mjs, paintfit.mjs	Card layout at every option count from 3 to 6, four viewports down to 360×640, last card visible.

Run order that has worked: `npx vitest run`, then `npm run build`, then `npx vite preview --port 4173`, then the scripts against it. The sweeps take 10–15 minutes and are worth it before any content change ships.

10. What is NOT built

Nothing scores a spoken answer. `src/assessment/speechScoring.ts` exports `STUB_SCORER`, which returns `{ scored: false, reason: 'no-scorer' }` for every take. A test asserts `SPEECH_SCORING_IS_STUB` so this cannot be forgotten.

This is a seam, not a hole in the plumbing: the UI is finished, the microphone works, takes are timed, and the flow handles an unscored answer correctly all the way to the parent screen. What is missing is the thing that turns audio into a verdict.

What it costs, concretely

Subject	Track	Cost
Oral Reading	Grade Level	Half the reading gate. Step 1 of the priority gate is the hardest block in the product and it runs on one measured subject instead of two.
Words Speaking	Grade Level	Produces observations nobody can act on.
Word Practice (1 move of 3)	Little Readers	One of only two subjects that place an under-7 , so a third of its evidence is unusable. This is why two of its three moves are tapped rather than spoken — the reference design was a microphone alone, which would have left an under-7 placed on Letter Sounds only.

The three options

Option	Gets you	Costs
Browser speech recognition (Web Speech API)	Ships today, free. The hooks already exist — <code>useReadingTracker.ts</code> uses it for read-along.	Mediocre with young voices and accents. Chrome-ish. Sends audio to a vendor.
Own Whisper endpoint	Accurate, private, works offline-ish, no vendor lock.	Money and infrastructure. Somebody owns a GPU or an inference bill.
Record, a teacher scores it	The most accurate option, and the only one that tells you what “good” actually sounds like for this age group.	Does not scale past the first cohort. Needs a queue and a teacher UI.

Recommendation for where this project is now — pre-launch, five teachers, no users: do the third. It is the only one that produces labelled data on real children, and you need that data to tune or train either of the other two later. Move to Whisper once the queue hurts.

Also pending, and smaller

- `LITTLE_READER_MAX_AGE = 6`, the reading-level rule and the program names are all config **pending teacher sign-off**. They are judgements about children, kept in one place each.
- The 60-second Sentence Writing timer is deliberately not built. It needs a teacher’s name against it before a clock goes near a child’s writing.
- Orange and purple are absent from the shape palette. Adding them means new brand hexes, which is a design-system decision.
- Sounds `q x y z v` have no picture word.

11. Running it

```
npm install
npm run dev # localhost:5173
npx vitest run # 157 tests
npm run build # -> dist/
npx vite preview --port 4173
```

Useful query parameters

The flow can be entered directly, which is how every browser script drives it:

```
http://localhost:4173/?name=Ada&age=5&grade=1 # Little Readers
http://localhost:4173/?name=Maya&age=10&grade=5 # Grade Level
```

Session state is in `localStorage`; clear it to start over.

Adding or replacing content

- Edit `src/content/questionBank.json`. Subject and tier live in the data — no engine change is needed.
- Generate items with a script that asserts its own rules, and keep the script. Every subject in this build has one; they caught real bugs (three tier-0 items parking the answer in the same slot, a colour guard that re-rolled per element).
- Then add tests that hold the *content*, not the code. See §9.
- Then look at it in a browser at four viewports. Every layout bug in §8 was found by looking, not by a test.

The one thing to keep

If you change nothing else, keep the rule that **an answer nothing could judge is recorded as unscored and told to the parent**. It is the difference between a placement you can defend and a number you made up.

Generated from the repository at branch `claude/olc-placement-assessment-y4lrpw`. The deeper narrative, including what was tried and abandoned, is in `README.md`.